

Attack Lab : 理解缓冲区溢出漏洞

布置日期：9月29日，周二

截止日期：10月8日，周四 23:59 EDT

最晚提交日期：10月11日，周日 23:59 EDT

1 简介

本实验需要针对两个存在不同安全漏洞的程序，总共发起五次攻击。完成本实验你将收获以下技能与知识：

- 深入学习当程序未对缓冲区溢出进行足够防护时，攻击者利用安全漏洞的多种途径。
- 借此更透彻地理解如何编写更具安全性的程序，并了解编译器和操作系统为降低程序易受攻击性而提供的防御特性。
- 对 x86-64 机器代码的栈机制及参数传递机制获得更深入的认知。
- 更深入地理解 x86-64 指令集的编码方式。
- 积累使用 GDB 和 OBJDUMP 等底层调试与逆向工具的实践经验。

注意：在本实验中，你将亲身体验用于利用操作系统和网络服务器安全缺陷的技术手段。我们的目的是帮助你掌握程序运行时的底层行为，并理解这些安全弱点的本质，以便你在编写系统级代码时能够有效避免它们。我们坚决反对并严禁使用任何其他形式的攻击来非法访问未经授权的系统资源。

建议复习《深入理解计算机系统》（CS:APP3e）教材中的第 3.10.3 节和第 3.10.4 节，作为完成本实验的重要参考资料。

2 实验准备与组织

与以往一样，这是一个个人独立完成的项目。你将针对为你量身定制生成的目标程序发起攻击。

2.1 获取实验文件

你可以通过在网络浏览器中访问以下网址来获取实验材料：

```
http://$Attacklab::SERVER_NAME:15513/
```

教师提示： `$Attacklab::SERVER_NAME` 是运行 attacklab 服务的服务器机器名。该项定义在 `attacklab/Attacklab.pm` 和 `attacklab/src/build/driverhdrs.h` 中。

服务器将构建你的实验文件，并以名为 `targetk.tar` 的归档压缩包形式返回给你的浏览器，其中 `k` 是你专属目标程序的唯一编号。

注意： 服务器构建并下载目标程序需要数秒时间，请耐心等待。

将 `targetk.tar` 文件保存到一个你打算开展工作的受保护 Linux 目录中。然后执行解压命令：`tar xvf targetk.tar`。解压后将生成一个名为 `targetk` 的目录，其中包含以下文件：

你应该只下载一套文件。如果因某种原因下载了多个 `target` 包，请挑选其中一个作为工作目标并彻底删除其余文件。

警告： 如果你在 Windows/PC 上解压 `targetk.tar`（例如使用 WinZip 等压缩软件），或者让浏览器自动解压，可能会导致可执行文件的 Linux 执行权限位丢失。

`targetk` 目录中的文件包括：

- `README.txt`：描述该目录内容的说明文本文件。
- `ctarget`：一个易受代码注入（Code-Injection）攻击的可执行程序。
- `rtarget`：一个易受返回导向编程（ROP）攻击的可执行程序。
- `cookie.txt`：一个 8 位十六进制代码，作为攻击中验证你唯一身份的标识符。
- `farm.c`：目标程序“gadget farm”的源代码，构造面向返回攻击时将从中提取代码片段。
- `hex2raw`：用于生成原始攻击字符串的实用辅助工具。

在随后的实验说明中，我们均假设你已将这些文件复制到受保护的本地目录中，并在该本地目录下运行这些程序。

2.2 重要考量与规则

以下是关于本实验有效解答的核心规则摘要。初次阅读本文档时，你可能对这些要点缺乏直观感受，但在此列出是为你后续动手操作时提供统一的权威规范参考：

- 你必须在一台与生成目标程序环境相同（或兼容）的 Linux 机器上完成实验。
- 你的解答**不得**利用恶意攻击去绕过程序本身的验证检测代码。具体而言，你在攻击字符串中包含供 `ret` 指令跳转的任何目标地址，必须且只能指向以下目的地之一：
 - 函数 `touch1`、`touch2` 或 `touch3` 的起始地址。
 - 你注入的代码的起始地址。
 - 来自 gadget farm 中的某个 gadget 的起始地址。
- 你只能从 `rtarget` 文件中提取 gadget，且其地址范围必须严格限定在函数 `start_farm` 与 `end_farm` 之间。

3 目标程序分析

`CTARGET` 和 `RTARGET` 都会从标准输入读取字符串。它们通过如下定义的 `getbuf` 函数来实现：

```
1 unsigned getbuf()
2 {
3     char buf[BUFFER_SIZE];
4     Gets(buf);
5     return 1;
6 }
```

函数 `Gets` 类似于 C 标准库函数 `gets` ——它从标准输入读取一行字符串（以换行符 `'\n'` 或 EOF 文件结束符为终止），并连同空字符结尾符 `\0` 一并存入指定目的地。在这段代码中可以看到，目标目的地是一个大小为 `BUFFER_SIZE` 字节的数组 `buf`。在生成专属你的实验程序时，`BUFFER_SIZE` 是一个与你版本绑定的编译期常量。

函数 `Gets()` 和 `gets()` 没有任何机制去判断目标缓冲区是否足够大以容纳输入的字符串。它们只是单纯盲目地拷贝字节序列，因而极易越界覆盖分配给目标缓冲区的内存空间。

如果用户输入的字符串足够短，`getbuf` 自然会正常返回 1，如下面正常执行示例所示：

```
unix> ./ctarget
```

```
Cookie: 0x1a7dd803
Type string: Keep it short!
No exploit. Getbuf returned 0x1
Normal return
```

而当你输入一段长字符串时，通常会发生严重错误：

```
unix> ./ctarget
Cookie: 0x1a7dd803
Type string: This is not a very interesting string, but it has the property...
Ouch! You caused a segmentation fault!
Better luck next time
```

（注意：示例中显示的 cookie 值与你的专属值不同。）程序 RTARGET 也会有同样的行为。如错误提示所示，缓冲区越界通常会破坏程序运行时的栈状态，从而引发内存非法访问段错误（Segmentation Fault）。你的任务就是精心构造输入给 CTARGET 和 RTARGET 的字符串，使其执行特定的非预期操作。这些精心制作的输入被称为**攻击字符串（exploit strings）**。

CTARGET 和 RTARGET 支持若干命令行参数：

- `-h`：打印可用命令行参数列表。
- `-q`：不向评分服务器发送评测结果。
- `-i FILE`：从指定文件读取输入，而非从标准输入键盘读取。

你的攻击字符串通常包含不属于可打印 ASCII 字符范围的原始字节。辅助工具 HEX2RAW 能将十六进制文本转换为原始字节串。详情参见附录 A。

关键注意事项：

- 攻击字符串中间**绝不能包含字节值 0x0a**，因为这是换行符（`'\n'`）的 ASCII 码。一旦 Gets 读到该字节，就会误认为输入已终止。
- HEX2RAW 接收由空格隔开的双位十六进制数。若要生成十六进制值为 0 的单字节，必须写作 `00`。要构造 32 位整数字字面量 `0xdeadbeef`，由于 x86-64 采用**小端字节序（little-endian）**，应向 HEX2RAW 传入 `ef be ad de`。

当你正确攻破某一关卡时，目标程序会自动向评分服务器发送通知。例如：

```
unix> ./hex2raw < ctarget.12.txt | ./ctarget
Cookie: 0x1a7dd803
Type string: Touch2! You called touch2(0x1a7dd803)
Valid solution for level 2 with target ctarget
PASSED: Sent exploit string to server to be validated.
NICE JOB!
```

阶段 (Phase)	目标程序	难度等级	攻击手法	目标函数	分值
1	CTARGET	1	代码注入 (CI)	touch1	10
2	CTARGET	2	代码注入 (CI)	touch2	25
3	CTARGET	3	代码注入 (CI)	touch3	25
4	RTARGET	2	返回导向编程 (ROP)	touch2	35
5	RTARGET	3	返回导向编程 (ROP)	touch3	5

注：CI 表示代码注入（Code Injection）；ROP 表示返回导向编程（Return-Oriented Programming）。

图 1：Attack Lab 各阶段攻击任务概览

服务器将验证你的攻击字符串以确保其真实生效，并更新 Attacklab 实时计分板，显示你的用户 ID（为保护隐私，以 target 编号呈现）已成功通关该阶段。

你可以通过在浏览器中访问以下网址查看实时排行榜：

```
http://$Attacklab::SERVER_NAME:15513/scoreboard
```

与炸弹实验（Bomb Lab）不同，本实验中尝试出错没有任何扣分惩罚。请放心大胆地尝试各种可能的攻击字符串测试 CTARGET 和 RTARGET。

重要提示：你可以在任意 Linux 机器上编写和调试你的解答，但为了向评分服务器成功提交结果，目标程序必须运行在教师指定的机器环境下：

教师说明：此处替换为你为实验在 buflab/src/config.c 中设定的合法域名或主机列表。

图 1 总结了本实验的五个关卡阶段。前三个阶段针对 CTARGET 进行代码注入（CI）攻击，后两个阶段则针对 RTARGET 进行返回导向编程（ROP）攻击。

4 第一部分：代码注入攻击 (Part I)

在前三个阶段中，你的攻击字符串针对的是 CTARGET。该程序在编译时被特别设置，保证每次运行时的栈位置固定不变，且栈内存区域被赋予了可执行权限。这些特性使得攻击者可以直接在栈中注入机器指令的字节编码并发起执行。

4.1 Level 1（难度等级 1）

在第 1 阶段中，你无需注入新代码。你的攻击字符串只需要重定向程序的控制流，使其跳转去执行一个已有的函数过程即可。

在 CTARGET 内部，函数 getbuf 是由一个名为 test 的函数调用的，其 C 代码如下：

```

1 void test()
2 {
3     int val;
4     val = getbuf();
5     printf("No exploit. Getbuf returned 0x%x\n", val);
6 }

```

当 `getbuf` 执行其 `return` 语句（第 5 行）时，程序正常情况下应恢复执行 `test` 函数（即第 5 行的 `printf`）。我们希望篡改这一行为。在 `ctarget` 文件中，存在一个名为 `touch1` 的函数，其 C 代码结构如下：

```

1 void touch1()
2 {
3     vlevel = 1;          /* 实验验证协议标记 */
4     printf("Touch1!: You called touch1()\n");
5     validate(1);
6     exit(0);
7 }

```

你的任务是：使得 `CTARGET` 在 `getbuf` 执行返回语句时，不是返回到 `test`，而是转去执行 `touch1` 的代码。注意，你的溢出攻击可能会破坏栈中与当前阶段无直接关联的数据，但这无关紧要，因为 `touch1` 最终会调用 `exit(0)` 正常终止程序。

提示与建议：

- 构建本关攻击字符串所需的一切信息，均可通过反汇编 `CTARGET` 获取。使用 `objdump -d ctarget` 即可获得反汇编代码。
- 核心思路是在栈中精心安排 `touch1` 入口起始地址的字节编码，恰好覆盖原本保存的返回地址，使 `getbuf` 尾部的 `ret` 指令将控制流转交给 `touch1`。
- 切记留意 x86-64 架构的小端字节序（Little-Endian）。
- 建议使用 GDB 并在 `getbuf` 的最后几条指令处单步跟入，验证程序执行流是否按预期跳转。
- `buf` 在 `getbuf` 栈帧内的相对位置和偏移量取决于编译期常量 `BUFFER_SIZE` 以及 GCC 的栈对齐/分配策略，你必须深入查看反汇编代码以确定准确的偏移量。

4.2 Level 2 (难度等级 2)

第 2 阶段要求在你的攻击字符串中注入一段极短的机器指令代码。

在 `ctarget` 文件中，包含名为 `touch2` 的函数，其 C 代码如下：

```

1 void touch2(unsigned val)
2 {
3     vlevel = 2;          /* 实验验证协议标记 */
4     if (val == cookie) {
5         printf("Touch2!: You called touch2(0x%.8x)\n", val);
6         validate(2);
7     } else {
8         printf("Misfire: You called touch2(0x%.8x)\n", val);
9         fail(2);
10    }
11    exit(0);
12 }

```

你的任务是让 `CTARGET` 在 `getbuf` 返回时去执行 `touch2` 的代码，而不是返回到 `test`。但与上一关不同的是，你必须让 `touch2` 认为你将专属的 `cookie` 作为第一个参数传给了它。

提示与建议：

- 你需要把注入的攻击机器码存放在缓冲区内，并将原本的返回地址覆盖为你注入代码的起始地址，使 `getbuf` 的 `ret` 指令跳转至你的注入代码处。
- 回想 x86-64 的函数调用约定：函数的第一个整型/指针参数是通过 `%rdi` 寄存器传递的。
- 你的注入代码应将 `%rdi` 寄存器赋值为你的专属 `cookie`，随后利用一条 `ret` 指令将控制权转交给 `touch2` 的第一条指令。
- **不要**尝试在注入代码中使用 `jmp` 或 `call` 指令。这些指令的目标地址采用相对偏移编码，手工计算极其繁琐复杂。请统一使用 `ret` 指令来转移控制流，即使并非真正从函数调用返回。
- 阅读附录 B，学习如何借助汇编工具生成自定义汇编指令序列的机器字节码。

4.3 Level 3 (难度等级 3)

第 3 阶段同样涉及代码注入攻击，但这次传递给目标函数的参数是一个字符串指针。

在 `ctarget` 文件中，包含了 `hexmatch` 和 `touch3` 两个函数，其 C 代码如下：

```
1 /* 将字符串与无符号数的十六进制表示形式进行比较 */
2 int hexmatch(unsigned val, char *sval)
3 {
4     char cbuf[110];
5     /* 使校验字符串的存放位置具有不可预测性 */
6     char *s = cbuf + random() % 100;
7     sprintf(s, "%.8x", val);
8     return strncmp(sval, s, 9) == 0;
9 }
```

```

10
11 void touch3(char *sval)
12 {
13     vlevel = 3;          /* 实验验证协议标记 */
14     if (hexmatch(cookie, sval)) {
15         printf("Touch3!: You called touch3(\"%s\")\n", sval);
16         validate(3);
17     } else {
18         printf("Misfire: You called touch3(\"%s\")\n", sval);
19         fail(3);
20     }
21     exit(0);
22 }

```

你的任务是让 `CTARGET` 在 `getbuf` 返回时执行 `touch3`，且必须让 `touch3` 接收到一个指向你 `cookie` 十六进制字符串表示的指针作为实参。

提示与建议：

- 你需要在攻击字符串中嵌入 `cookie` 的 ASCII 字符串表示。该字符串由 8 个十六进制字符构成（从最高有效位到最低有效位顺序排列），且**不包含**前缀 `"0x"`。
- 回顾 C 语言字符串规范：字符串是以值为 0 的字节（`\0`）结尾的字节序列。在任意 Linux 终端中运行 `man ascii` 可速查各字符对应的 ASCII 十六进制编码。
- 你的注入代码应将 `%rdi` 寄存器设置为该字符串在内存中的有效起始地址。
- 当 `touch3` 进一步调用 `hexmatch` 和 `strncmp` 时，这些函数会在栈上压入自己的栈帧数据，进而**覆盖原本 `getbuf` 所使用的部分栈内存区域**。因此，你必须极其审慎地规划并挑选放置 `cookie` 字符串的内存位置，防止其在被比对前惨遭冲刷覆盖！

5 第二部分：返回导向编程 (Part II: ROP)

对程序 `RTARGET` 发起代码注入攻击要困难得多，因为该程序采用了两项现代操作系统的主流安全防护技术：

- **地址空间布局随机化 (ASLR)**：每次运行时栈的位置都会随机变动，使得攻击者无法预先获知所注入代码的确切内存地址。
- **栈不可执行 (NX / DEP)**：将栈所在的内存段标记为只读/可读写但不可执行。即使攻击者成功将 PC 指针指向栈中的注入代码，CPU 也会立刻触发段错误异常并终止运行。

幸运的是，安全研究人员提出了利用程序中现有既有代码来完成恶意任务的强大技术，而不必注入新代码。其中最通用的形式便是**返回导向编程 (Return-Oriented Programming, ROP)** [1, 2]。ROP 的核心策略是在现有程序的二进制代码中，搜索并定位由一条或多条指令后紧跟一条 `ret` 指令构成的微型字节片段。这种代码片段被称为 **gadget**（小构件）。

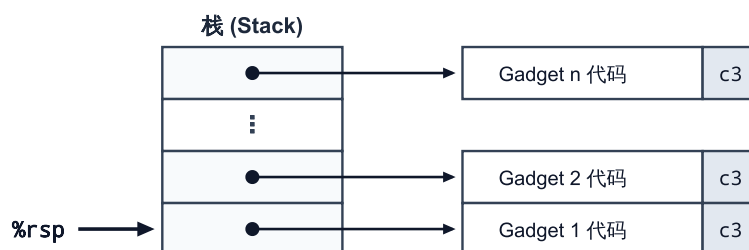


图 2：组织 gadget 链式序列执行。字节值 `0xc3` 代表 `ret` 指令的机器码。

图 2 展示了如何巧妙布置栈空间来连续触发执行由 n 个 gadget 组成的链条。在图中，栈内存储了一连串 gadget 的入口地址。每个 gadget 包含若干条机器指令字节，并以 `0xc3`（即 `ret` 指令编码）结尾。当程序在这种栈配置下执行 `ret` 指令时，将触发连锁执行——每个 gadget 末尾的 `ret` 都会从栈顶弹出下一个 gadget 的地址并跳转执行。

Gadget 可以直接借用编译器正常生成的汇编代码（尤其是函数末尾处的指令）。但实际中这类直接可用的片段功能有限，不足以实现复杂运算（例如极少有编译后的函数会将 `popq %rdi; ret` 作为收尾）。幸运的是，在 x86-64 这种变长字节指令集中，我们常常可以通过错位截取长指令内部的连续字节序列，挖掘出意想不到的全新有效指令！

例如，某版本 `rtarget` 中包含由以下 C 函数编译生成的代码：

```
void setval_210(unsigned *p)
{
    *p = 3347663060U;
}
```

从源码看该函数对攻击毫无用处。但在反汇编机器代码中，它却暴露出极其有趣的字节序列：

```
0000000000400f15 <setval_210>:
  400f15: c7 07 d4 48 89 c7    movl $0xc78948d4, (%rdi)
  400f1b: c3                  retq
```

注意观察字节序列 `48 89 c7`，这恰好是指令 `movq %rax, %rdi` 的机器编码！（参见图 3A 获取常见 `movq` 编码）。其后紧接着字节 `c3`（即 `ret`）。该函数起始于 `0x400f15`，该有用字节片段起始于其第 4 个字节。因此，这里隐蔽潜藏着一个起始地址为 `0x400f18` 的 gadget，它能把 `%rax` 的 64 位值无损拷贝至 `%rdi`！

你的 `RTARGET` 包含许多类似于 `setval_210` 的函数，它们统一聚集存放在被称为 **gadget farm**（gadget 农场）的内存区域。你的任务就是在该农场中识别挖掘出有用的 `gadgets`，并用它们拼装发起类似于阶段 2 和阶段 3 的攻击效果。

重要限制：在你的 `rtarget` 二进制程序中，`gadget farm` 的代码范围被函数 `start_farm` 和 `end_farm` 严格标界。切勿尝试从程序其他区域截取构造 `gadget`。

5.1 Level 2（难度等级 2 / 对应 Phase 4）

在第 4 阶段中，你需要复现第 2 阶段（调用 `touch2(cookie)`）的攻击效果，但这次是在启用防护的 `RTARGET` 上，纯粹利用 `gadget farm` 中的 `gadgets` 来拼装实现。你可以使用仅包含以下指令类型且仅使用前 8 个通用寄存器（`%rax - %rdi`）的 `gadgets`：

- `movq`：机器编码对照参见图 3A。
- `popq`：机器编码对照参见图 3B。
- `ret`：单字节编码为 `0xc3`。
- `nop`：单字节编码为 `0x90`（空操作，仅使 PC 指针递增 1）。

提示与建议：

- 完成本关所需的全部 `gadgets` 均可在由 `start_farm` 到 `mid_farm` 之间的代码区域内找到。
- 本阶段攻击**最少仅需 2 个 `gadgets`**即可完美达成。
- 当 `gadget` 中包含 `popq` 指令时，它会从栈顶弹出一个 64 位字并存入指定寄存器。因此，你的攻击字符串应当由 **`gadget` 入口地址与携带的数据（如 `cookie`）交替交织构成**。

5.2 Level 3（难度等级 3 / 对应 Phase 5）

在向第 5 阶段发起挑战前，不妨稍作停顿，盘点你目前已取得的辉煌成果：在第 2、3 阶段中，你迫使程序执行了你自主编写的任意机器码；若 `CTARGET` 是联网服务器，你已能实施远程代码执行。在第 4 阶段中，你更是成功攻破了现代系统最核心的两道防线（ASLR 与 NX 栈不可执行），通过拼接既有代码片段开辟了新控制流。此时你已经获得了本实验 100 分中的 95 分！这是一个非常优秀的成绩。若你近期还有其他紧迫学业任务，完全可以就此打住。

第 5 阶段要求在 `RTARGET` 上利用 ROP 链调用 `touch3`，并传入指向你的 `cookie` 字符串的有效指针。这听起来似乎只是调用 `touch2` 的微调版，但我们在题目设计上做了深度升级，使得获取运行时的字符串指针难度陡增。此外，第 5 阶段仅占 5 分，这与其所需的思考与调试时间并不成正比。将其视作为志在登峰造极、挑战极限的同学准备的附加题！

A. movq 指令的字节编码

movq S, D

源 (S)	目的操作数 (D)							
	%rax	%rcx	%rdx	%rbx	%rsp	%rbp	%rsi	%rdi
%rax	48 89 c0	48 89 c1	48 89 c2	48 89 c3	48 89 c4	48 89 c5	48 89 c6	48 89 c7
%rcx	48 89 c8	48 89 c9	48 89 ca	48 89 cb	48 89 cc	48 89 cd	48 89 ce	48 89 cf
%rdx	48 89 d0	48 89 d1	48 89 d2	48 89 d3	48 89 d4	48 89 d5	48 89 d6	48 89 d7
%rbx	48 89 d8	48 89 d9	48 89 da	48 89 db	48 89 dc	48 89 dd	48 89 de	48 89 df
%rsp	48 89 e0	48 89 e1	48 89 e2	48 89 e3	48 89 e4	48 89 e5	48 89 e6	48 89 e7
%rbp	48 89 e8	48 89 e9	48 89 ea	48 89 eb	48 89 ec	48 89 ed	48 89 ee	48 89 ef
%rsi	48 89 f0	48 89 f1	48 89 f2	48 89 f3	48 89 f4	48 89 f5	48 89 f6	48 89 f7
%rdi	48 89 f8	48 89 f9	48 89 fa	48 89 fb	48 89 fc	48 89 fd	48 89 fe	48 89 ff

B. popq 指令的字节编码

操作	寄存器 (R)							
	%rax	%rcx	%rdx	%rbx	%rsp	%rbp	%rsi	%rdi
popq R	58	59	5a	5b	5c	5d	5e	5f

C. movl 指令的字节编码

movl S, D

源 (S)	目的操作数 (D)							
	%eax	%ecx	%edx	%ebx	%esp	%ebp	%esi	%edi
%eax	89 c0	89 c1	89 c2	89 c3	89 c4	89 c5	89 c6	89 c7
%ecx	89 c8	89 c9	89 ca	89 cb	89 cc	89 cd	89 ce	89 cf
%edx	89 d0	89 d1	89 d2	89 d3	89 d4	89 d5	89 d6	89 d7
%ebx	89 d8	89 d9	89 da	89 db	89 dc	89 dd	89 de	89 df
%esp	89 e0	89 e1	89 e2	89 e3	89 e4	89 e5	89 e6	89 e7
%ebp	89 e8	89 e9	89 ea	89 eb	89 ec	89 ed	89 ee	89 ef
%esi	89 f0	89 f1	89 f2	89 f3	89 f4	89 f5	89 f6	89 f7
%edi	89 f8	89 f9	89 fa	89 fb	89 fc	89 fd	89 fe	89 ff

D. 2 字节功能性 nop 指令的字节编码

操作	寄存器 (R)			
	%al	%cl	%dl	%bl
andb R, R	20 c0	20 c9	20 d2	20 db
orb R, R	08 c0	08 c9	08 d2	08 db
cmpb R, R	38 c0	38 c9	38 d2	38 db
testb R, R	84 c0	84 c9	84 d2	84 db

图 3：相关指令的机器字节编码对照表（所有数值均为十六进制）

攻破第 5 阶段，你需要使用 `rtarget` 中位于 `start_farm` 与 `end_farm` 整个范围内的 gadgets。除了第 4 阶段用到的指令，此扩展农场还包含多种 `movl` 指令的编码（如图 3C 所示）。此外，该区域还潜藏着充当“功能性 `nop`（functional `nop`）”的 2 字节指令——即它们不改变任何寄存器数值或内存数据。如图 3D 所示，诸如 `andb %al, %al` 等指令只会对部分寄存器的最低字节进行位运算，但丝毫不会改变其寄存器数值。

提示与建议：

- 请务必回顾教材第 183 页关于 `movl` 指令对 64 位寄存器高 4 字节的清零副作用（即写低 32 位寄存器时，高 32 位会被硬件自动清零）。
- 官方参考解答总共使用了 8 个 gadgets（部分 gadget 有所复用）。

祝你好运，享受底层漏洞攻防的乐趣！

附录 A：使用 HEX2RAW 工具

HEX2RAW 接收十六进制格式的文本字符串作为输入。在此格式下，每个字节由两个十六进制字符表示。例如字符串 `"012345"` 在十六进制输入格式中写作 `30 31 32 33 34 35 00`（回顾可知字符 `'0'` 的 ASCII 码是 `0x30`，且 C 字符串以空字节 `00` 结尾）。

输入给 HEX2RAW 的十六进制字符之间必须以空白符（空格或换行）分隔。强烈建议在编写复杂的攻击文本时，用换行符将攻击 payload 的不同逻辑段落清晰隔开。

HEX2RAW 支持 C 语言风格的块级注释 `/* ... */`，方便你为攻击串标注说明。例如：

```
48 c7 c1 f0 11 40 00 /* mov $0x40011f0, %rcx */
```

请确保在注释的起始符 `/*` 和结束符 `*/` 两侧均保留空格，以便解析器将其正确识别为空白并忽略。

若将十六进制攻击文本保存在 `exploit.txt` 中，可以通过以下几种方式将其转换并输入给 `CTARGET` 或 `RTARGET`：

1. 利用管道直接将数据传递给程序：

```
unix> cat exploit.txt | ./hex2raw | ./ctarget
```

2. 先将原始字节存入文件，再通过 I/O 重定向传入：

```
unix> ./hex2raw < exploit.txt > exploit-raw.txt
unix> ./ctarget < exploit-raw.txt
```

这种方法在 GDB 调试会话中同样适用：

```
unix> gdb ctarget
(gdb) run < exploit-raw.txt
```

3. 将原始字节文件作为命令行参数传入：

```
unix> ./hex2raw < exploit.txt > exploit-raw.txt
unix> ./ctarget -i exploit-raw.txt
```

在 GDB 中同样支持使用 `-i` 参数：`(gdb) run -i exploit-raw.txt`。

附录 B：生成指令字节码

利用 GCC 作为汇编器、OBJDUMP 作为反汇编器，可以极为便捷地生成任意自定义汇编指令序列对应的机器字节码。例如，假设你创建了一个名为 `example.s` 的文件，包含以下汇编代码：

```
# 手写汇编代码示例
pushq    $0xabcdef      # 将数值压入栈
addq     $17, %rax       # 将 17 加到 %rax
movl     %eax, %edx      # 将低 32 位拷贝到 %edx
```

汇编源码中可以混合包含指令与数据。字符 `#` 右侧的所有内容均被视作单行注释。

现在你可以对其进行编译并反汇编：

```
unix> gcc -c example.s
unix> objdump -d example.o > example.d
```

生成的 `example.d` 文件将包含如下反汇编内容：

```
example.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <.text>:
   0: 68 ef cd ab 00      pushq $0xabcdef
   5: 48 83 c0 11         add    $0x11,%rax
   9: 89 c2               mov    %eax,%edx
```

底部的输出展示了从汇编语言翻译而来的机器代码。每行最左侧为十六进制表示的相对起始地址（从 0 开始），冒号 `:` 后面的十六进制数字即为对应的机器指令字节码。例如，指令 `push $0xabcdef` 对应的机器字节序列为 `68 ef cd ab 00`。

从反汇编文件中，我们提取出该段指令的完整字节序列：

```
68 ef cd ab 00 48 83 c0 11 89 c2
```

随后便可将该十六进制文本输入给 `HEX2RAW`，生成注入目标程序的原始字节序列。或者，你也可以直接编辑整理 `example.d`，剔除地址与无关符号，保留 C 风格注释以增强可读性：

```
68 ef cd ab 00    /* pushq $0xabcdef */
48 83 c0 11      /* add  $0x11,%rax */
89 c2           /* mov  %eax,%edx */
```

这也是完全合法的 `HEX2RAW` 输入格式，非常便于在调试过程中检查与维护攻击代码。

参考文献 (References)

1. R. Roemer, E. Buchanan, H. Shacham, and S. Savage. *Return-oriented programming: Systems, languages, and applications*. ACM Transactions on Information and System Security, 15(1):2:1–2:34, March 2012.
2. E. J. Schwartz, T. Avgerinos, and D. Brumley. *Q: Exploit hardening made easy*. In USENIX Security Symposium, 2011.